

Lecture 1.

Models of Computation

Software Development & Python
Ivan Solomatin
MIPT

Outline

- What does it mean “to compute”?
- Finite-state machines
- Pushdown automation
- Turing machines
- Universal computation
- Limits of computation

What does it mean “to compute”?

Definition

What does it mean “to compute”?

Definition

Input: two numbers a and b

- Read a
- Read b
- Add them
- Return the result

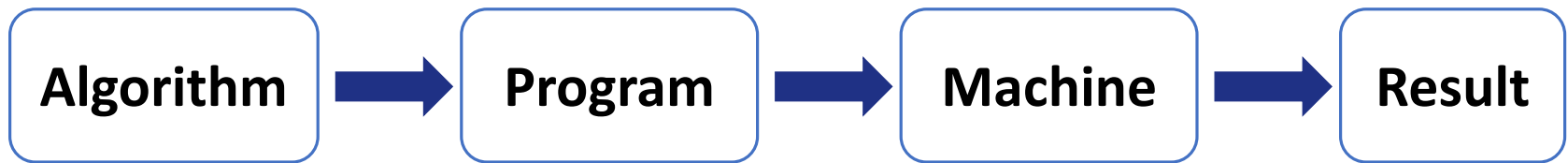
```
(defun add (a b)
  (+ a b))
```

```
def add(a, b):
    return a + b
```

```
int add(int a, int b)
{
    return a + b;
}
```

Same algorithm — different representation.

What does it mean “to compute”?



The algorithm is an idea. A program is its representation.
A machine defines how that representation is executed.

A model of computation.

Definition

A **model of computation** is a model that describes how an output of a mathematical function is computed given an input. A model of computation describes how units of computations, memories, and communications are organized. (Wiki)

A model of computation defines:

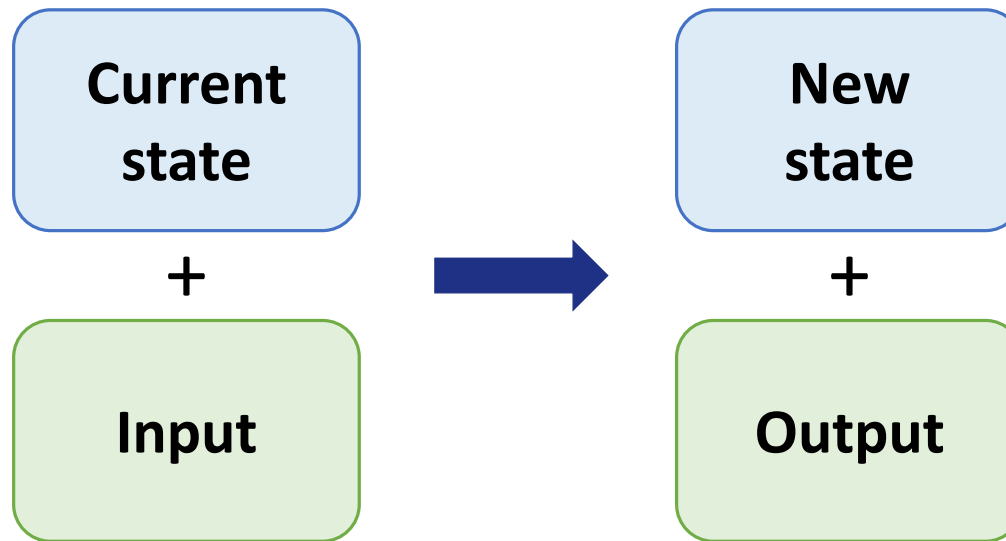
- What information the machine can store;
- What elementary operations it can perform;
- How it chooses the next operation;
- When the computation stops.

We can study computation without committing to a physical computer. We will do that during this lecture.

Finite-State Machines

Finite-State Machines

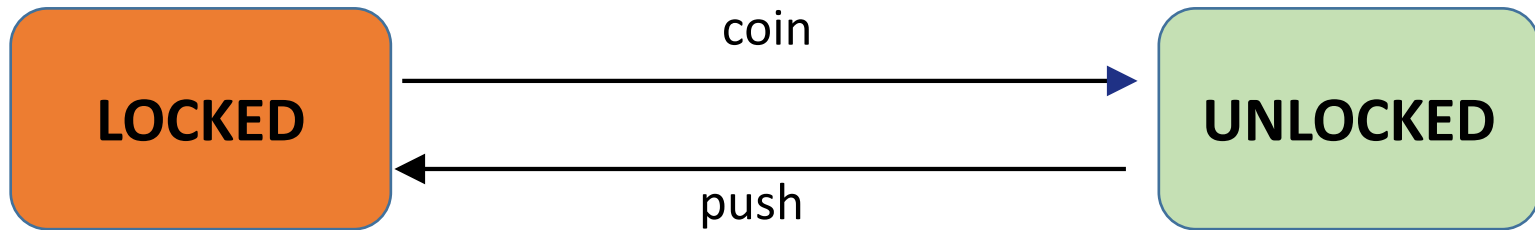
The simplest useful computational model.



Finite automaton = finite set of states + transition rules.

Finite-State Machines

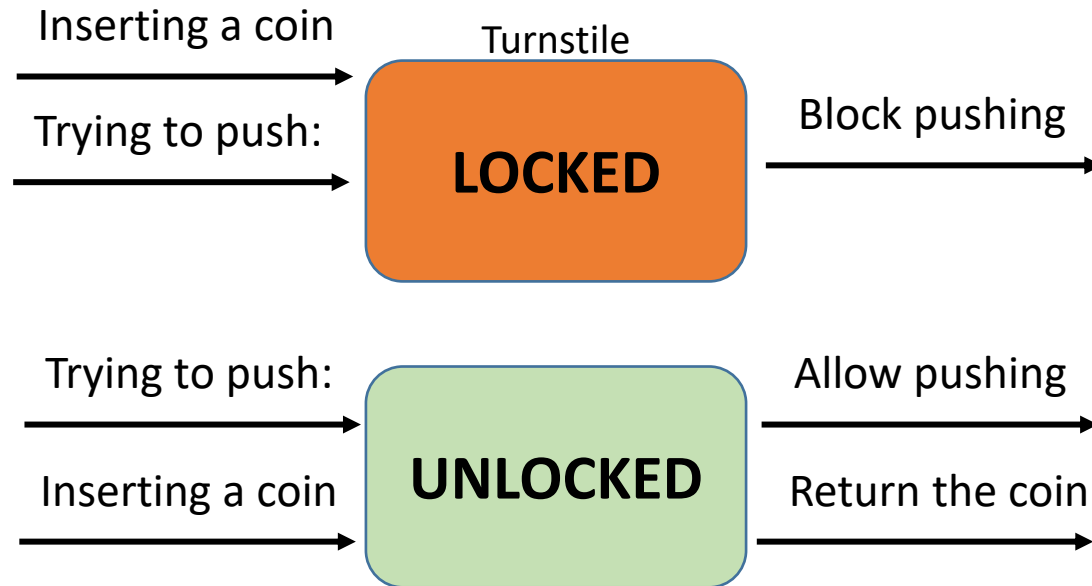
Example: Turnstile



The same input can have different effect in different states.

Finite-State Machines

Example: Turnstile



The same input can mean different things in different states.

Finite-State Machines

Example: Turnstile

Finite automaton = finite set of states + transition rules.

Set of states: $S = \{\text{LOCKED}, \text{UNLOCKED}\}$

Transition rules:

State	Input	Output	Next state
LOCKED	coin	NONE	UNLOCKED
LOCKED	push	Block pushing	LOCKED
UNLOCKED	push	Allow pushing	LOCKED
UNLOCKED	coin	Return the coin	UNLOCKED

State summarizes everything from the past that matters for the future. In fact, a state plays the role of memory.

But the amount of memory is fixed by the number of states.⁹

Finite-State Machines

Applications

User interfaces

Network protocols

Regular expressions

Game Dev

Hardware controllers

Simple parsers

Finite state is often exactly the right abstraction.

Pushdown automation

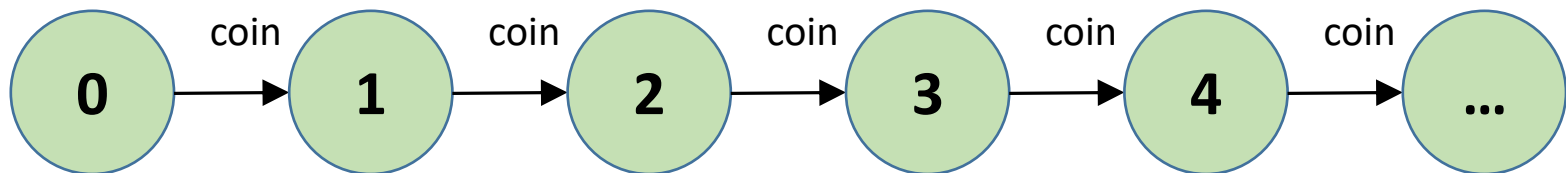
Pushdown automation

Limitation of Finite-State Machines.

What if we want our turnstile to allow the following scenario:

- Insert 3 coins
- 3 people enter

To do that we should *remember* how many coins were inserted.
Can an FSM remember an arbitrary number?



Unbounded count $\Rightarrow$ unbounded number of states.

Pushdown automation

Limitation of Finite-State Machines.

State	Input	Output	Next state
0	coin	NONE	1
0	push	Block pushing	0
1	coin	NONE	2
1	push	Allow pushing	0
2	coin	NONE	3
2	push	Allow pushing	1
...	...	...	...

Pushdown automation

Example: balanced parentheses.

Can a finite-state machine recognize correctly nested parentheses of arbitrary depth?

(() (())) **Correct**
depth: 0 1 2 1 2 3 2 1 0

(())) **Incorrect**
depth: 0 1 2 1 0 -1

(()) ((**Incorrect**
depth: 0 1 2 1 0 1 2

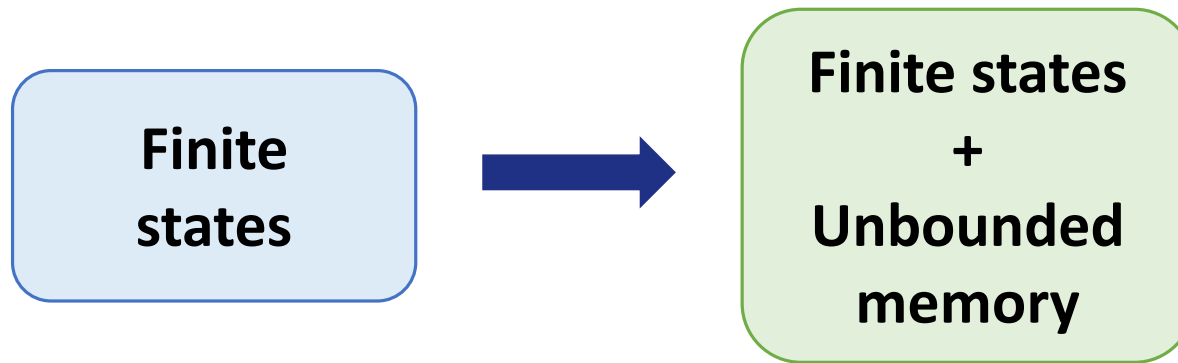
Pause the video for 20 seconds.

What information must the machine remember?

How many states do we need to handle sequences with up to 1000 parenthesis? How many rules?

Pushdown automation

Adding the memory to FSM



Some tasks require memory whose size grows with the input.

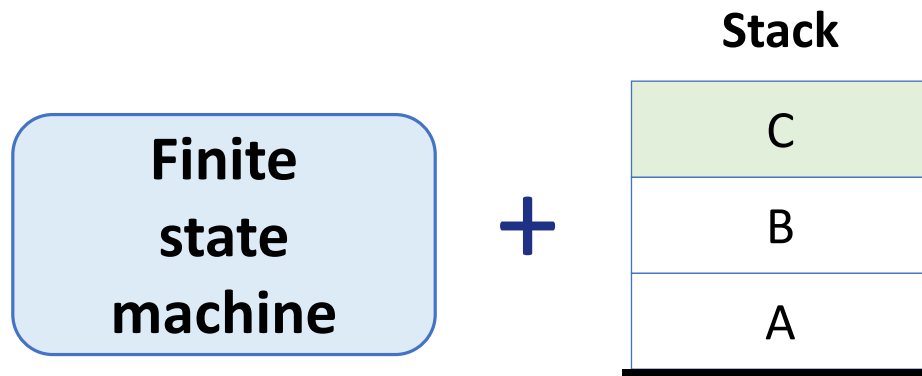
So let us extend the machine.

Pushdown automation

Adding the memory to FSM

Pushdown automaton:

Finite state machine + stack memory



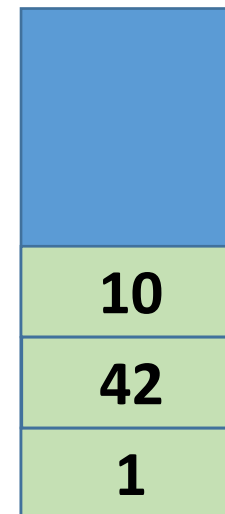
Stack is a data structure with following operations defined:

push(x) – place x in top of the stack

front() – returns value from top of the stack

pop() – returns value from top of the stack and removes it

len() – returns number of elements in stack



- > push(1)
- > push(5)
- > push(10)
- > pop() → 10
- > pop() → 5
- > push(42)
- > pop() → 42
- > pop() → 1

Pushdown automation

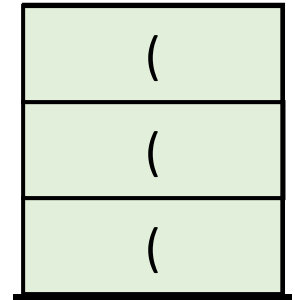
Example: balanced parentheses.

Input

(() (()))

push push pop push push pop pop pop

Stack



A stack handles nested structure naturally

This is why stacks appear in parsers and function call stacks.

The Turing Machine

The Turing Machine

A minimal model of general-purpose computation.

Alan Turing, 1936. *“On Computable Numbers, with an Application to the Entscheidungsproblem”*.

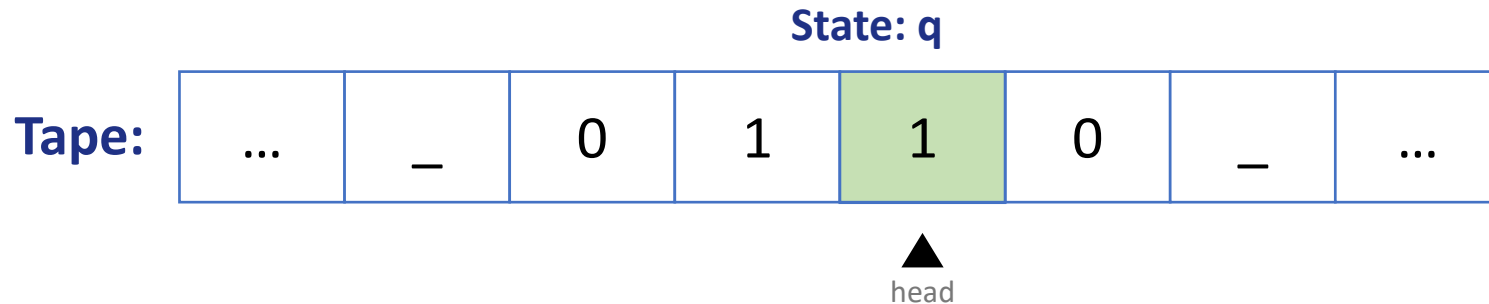
The goal was not to design a practical computer.

The goal was to formalize the idea of an effective mechanical procedure.

This was before modern general-purpose electronic computers.

The Turing Machine

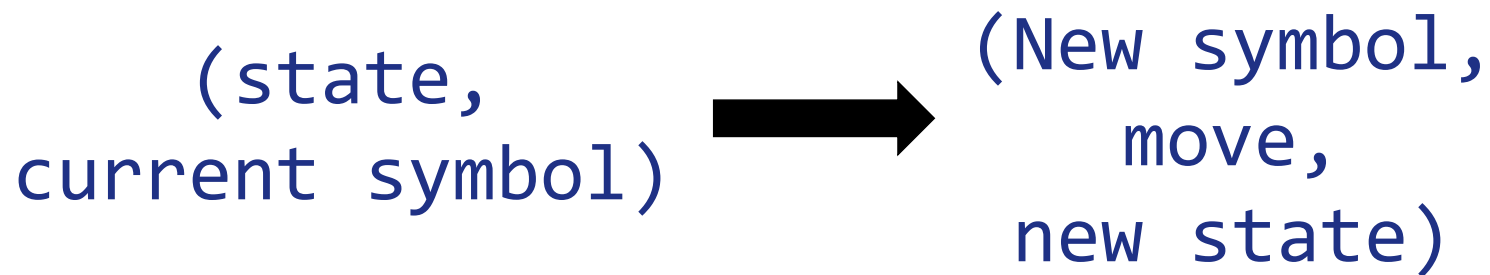
Idea.



- look at the current symbol (pointed by head);
- remember a finite state (like in FSM);
- write a symbol to current position;
- move one cell left or right or stay on the same place (R, L, S);
- change state.

The Turing Machine

Transition rules.

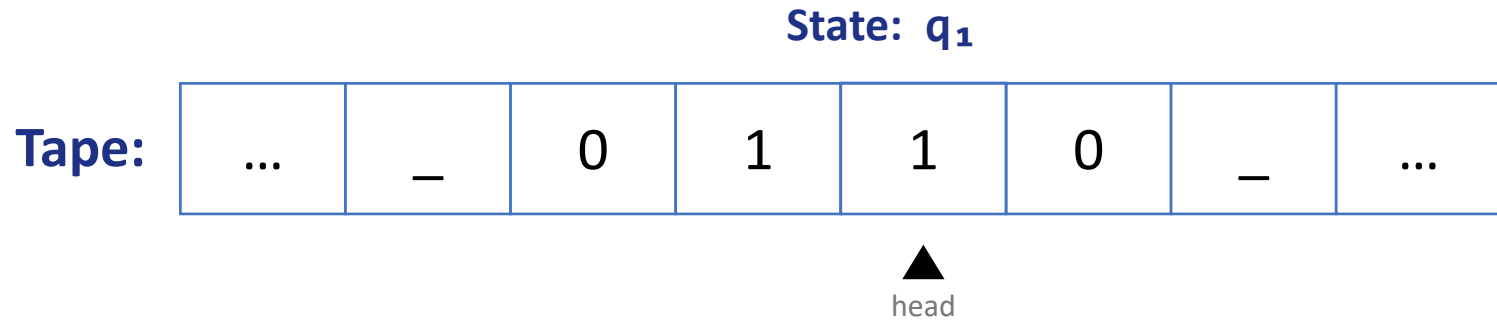


Example: $(q_0, 1) \rightarrow (\emptyset, R, q_1)$

One rule describes one elementary step of computation.

The Turing Machine

One transition.



$$(q_0, 1) \rightarrow (\emptyset, R, q_1)$$

write 0
move R
state q_1

The machine has no “high-level” operations — only local transitions.

The initial state is usually called q_0 .

The final state is called *HALT*.

The Turing Machine

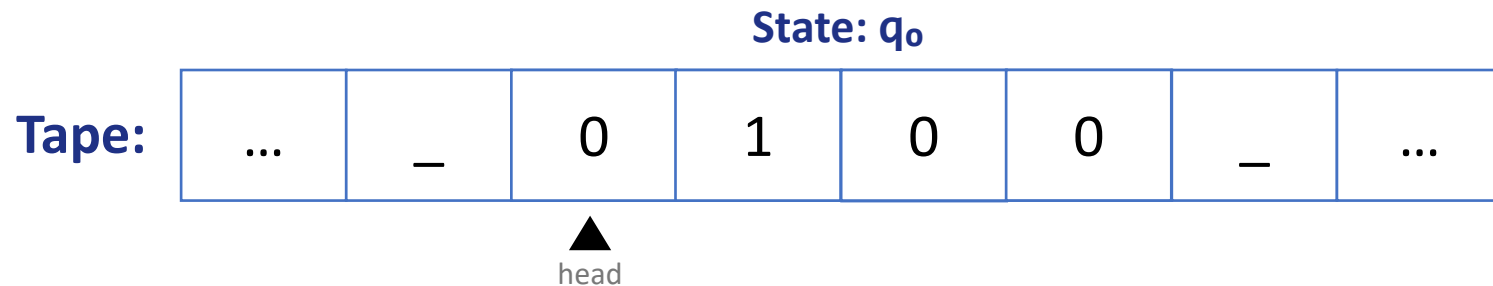
Example: binary inversion

Input: **1011**

Output: **0100**

How can a Turing machine do this?

Pause the video and design the algorithm first.



$(q_0, 1) \rightarrow (0, R, q_0)$

$(q_0, 0) \rightarrow (1, R, q_0)$

$(q_0, _) \rightarrow (_, S, \text{HALT})$

The Turing Machine

Example: binary increment

Input: **1011**

Output: **1100**

$$11 + 1 = 12$$

How can a Turing machine do this?

Pause the video and design the algorithm first.

1011 → 1010 → 1000 → 1100

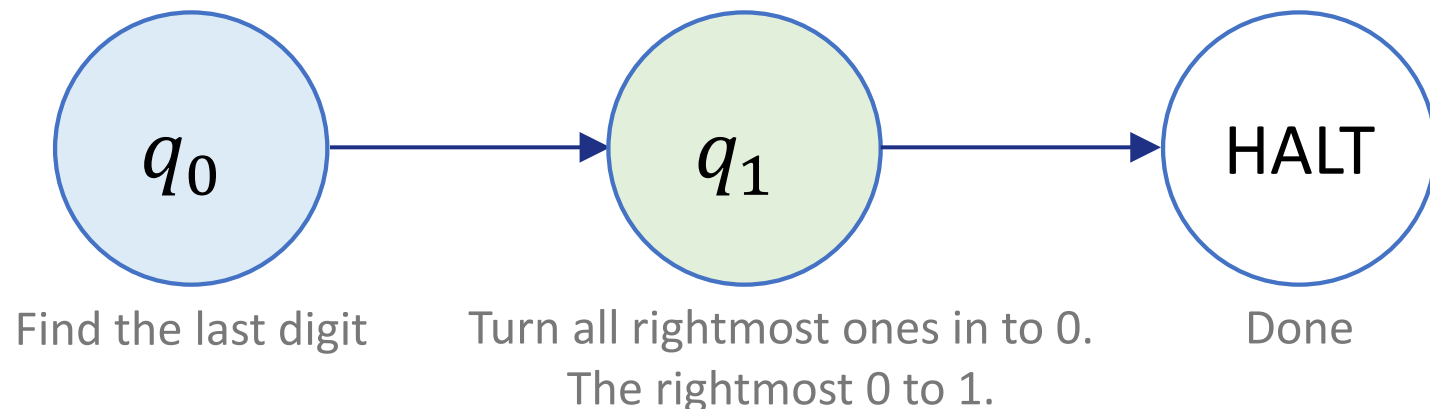
The Turing Machine

Example: binary increment

1011 → 1010 → 1000 → 1100

- Move to the last character in the input.
- If the current digit is 1: write 0 and carry to the left.
- If the current digit is 0: write 1 and stop.
- If we pass the left edge: write a new leading 1.

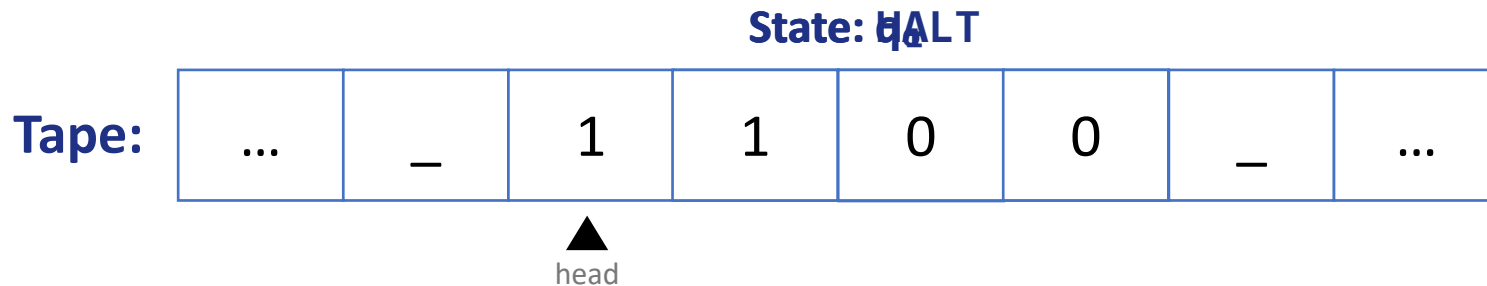
Which states do we need?



A state often tells us which stage of the algorithm we are in.

The Turing Machine

Example: binary increment



$(q_0, 1) \rightarrow (1, R, q_0)$

$(q_0, 0) \rightarrow (0, R, q_0)$

$(q_0, _) \rightarrow (_, L, q_1)$

$(q_1, 1) \rightarrow (0, L, q_1)$

$(q_1, 0) \rightarrow (1, S, \text{HALT})$

$(q_1, _) \rightarrow (1, S, \text{HALT})$

The Turing Machine

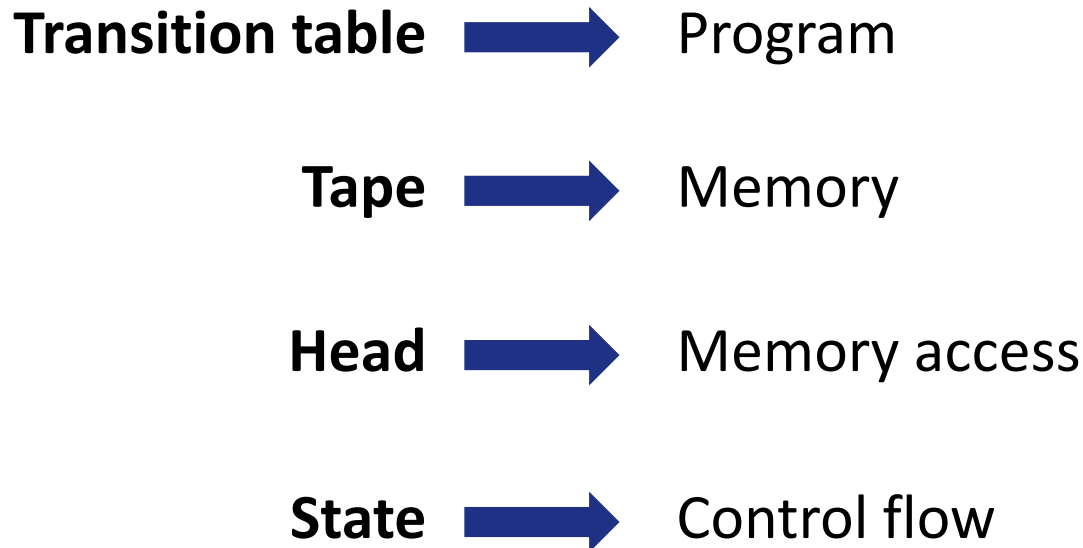
Example: binary increment. Transition table.

State	Read	Write	Move	Next State
q_0	0	0	R	q_0
q_0	1	1	R	q_0
q_0	—	—	L	q_1
q_1	0	1	S	HALT
q_1	1	0	L	q_1
q_1	—	1	S	HALT

Transition table = program.

The Turing Machine

We just wrote a program



The correspondence is conceptual, not architectural.

The Turing Machine

Why is this tiny machine so powerful?

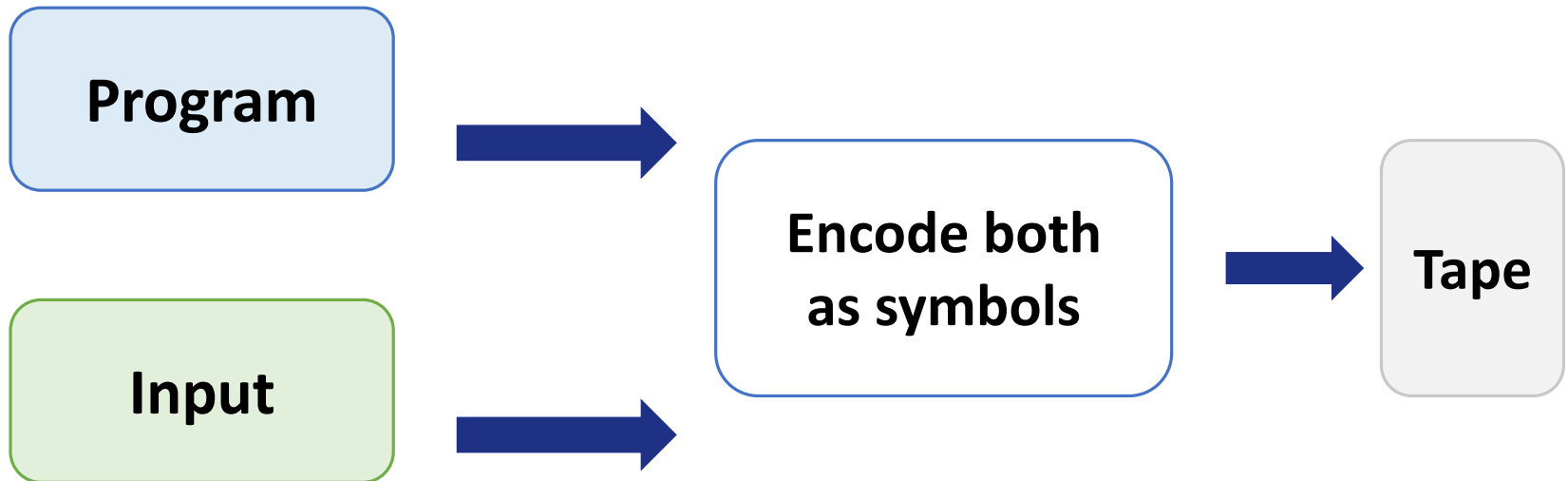
- The tape can grow as much as the computation needs.
- Numbers, strings, arrays and program code can all be encoded as symbols.
- A finite alphabet can represent arbitrarily large data.
- Complex behavior emerges from repeated simple transitions.

Simple machine $\neq$ simple computation.

Universal Computation

Universal Computation

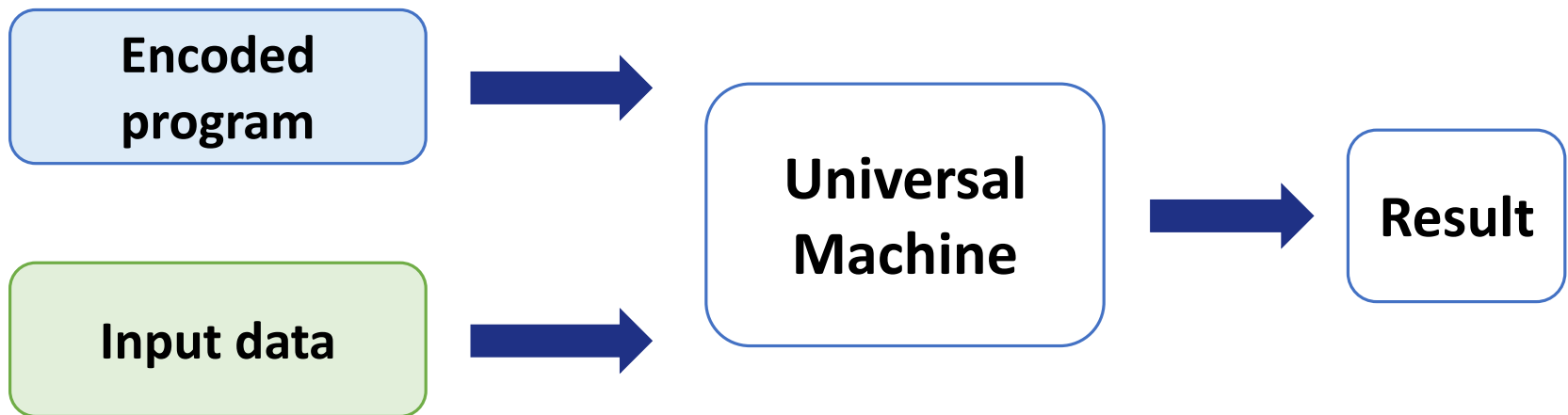
Can one machine execute every program?



The program itself can be represented in memory.

Universal Computation

Can one machine execute every program?



A single machine can simulate any other Turing machine.

This is the core idea of a programmable general-purpose computer.

Universal Computation

Definitions

Church–Turing thesis:

Everything that is effectively computable can be computed by a Turing machine.

Definition (Turing completeness):

A model of computation is called

Turing-complete or **computationally universal** if it can be used to simulate any Turing machine given enough time and memory.

Universal Computation

Turing completeness

A computational system is Turing complete if it can simulate any Turing machine, given enough time and memory.

Python

C

JavaScript

**Lambda
calculus**

Same class of computable functions — very different ways to describe them.

Universal Computation

Turing completeness

Excel
(Google spreadsheets)

Formula in K18: `=ЕСЛИ(K14<>0;ЕСЛИ(ИНДЕКС(rules;K14+1;4+K13*3)=0;ИНДЕКС(rules;K14+1;3+K13*3);0);ЕСЛИ(J14<>0;ЕСЛИ(ИНДЕКС(rules;J14+1;4+J13*3)=1;ИНДЕКС(rules;J14+1;3+J13*3);0);ЕСЛИ(J14<>0;ЕСЛИ(ИНДЕКС(rules;J14+1;4+J13*3)=1;ИНДЕКС(rules;J14+1;3+J13*3);0);0)))`

	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
12															
13	2	2	2	2	2	1	0	1	1	1	2	2	2	2	
14	0	0	0	0	0	1	0	0	0	0	0	0	0	0	
15															
16															
17	2	2	2	2	2	1	0	1	1	1	2	2	2	2	
18	0	0	0	0	0	0	1	0	0	0	0	0	0	0	
19															
20															

Minecraft redstone



Instead of launching Doom on everything, you may launch TM to show it can compute everything 😊



Universal Computation

Turing completeness

Then why do we need Python?

Turing completeness tells us what can be expressed,
not what is *convenient, safe, or efficient*.

Programming languages are interfaces for humans.

Limits of Computation

Universal computations do not give absolute power.

Limits of Computation

The Halting Problem

Imagine a
function:

halts(program, input)

It should always
answer:

True — the program eventually stops

False — the program runs forever

**Can such an algorithm exist for every possible
program and input?**

Limits of Computation

The Halting Problem

The paradox idea:

```
def g() -> None:
    if halts(g):
        loop_forever()
```

halts → loops

Contradiction: no general halting decider exists.

Limits of Computation

Computable does not mean practical

Not computable

Computability

**Computable but
prohibitively
expensive**

Complexity

**Efficiently
computable**

Algorithms course

Different questions — both fundamental.

$$P \stackrel{?}{=} NP$$

Conclusion

Conclusion

Other models of computation

**Turing
machines**

**Lambda
calculus**

**Recursive
functions**

**Register
machines**

These very different models turn out to have the same computational power.

Church–Turing Thesis

Everything that is effectively computable can be computed by a Turing machine.

Conclusion

What should you remember?

- Computation can be studied independently of physical computers.
- A computational model specifies memory, operations, control and termination.
- Finite-state machines have finite memory.
- A Turing machine adds simple but unbounded memory.
- A universal machine can treat programs as data.
- Some precisely stated problems cannot be solved by any algorithm.

Turing complete $\neq$ fast, convenient, or practical.

Conclusion

Thank you for watching!